Pre-Reflection

After reading through the updated requirements, I realized I needed to transition from using a 2D array to a dynamic **LinkedList**. I also needed to integrate a **HashMap** for fast searching, a **Queue** for car wash processing, and **ArrayList** for sorting the cars. I also had to apply object-oriented principles by using an **abstract Car class** with **BasicCar** and **PremiumCar** subclasses.

Classes and Methods

Class	Description
Car (abstract)	Holds shared attributes (brand, type, year, price, position) and declares `displayInfo()`
BasicCar	Extends Car; Implements `displayInfo()`
PremiumCar	Extends Car; Implements `displayInfo()`
Node	Represents a node in the linked list, containing a Car
VendingMachine	Manages linked list of cars, searching, sorting, and selling cars
CarWash	Manages car wash queue using linked list

Tasks and User Stories

User Story	Tasks	Time
Store Cars in Vending Machine (File Input)	Create LinkedList Node class, Read file, Validate slot, Add to LinkedList	2 Days
Display Car Inventory	Implement linked list traversal to display cars	0.5 Day
Sort Inventory (Price/Year)	Move cars to ArrayList, sort using Collections.sort()	1 Day
Retrieve Car by Position	Build HashMap from LinkedList, search by key	1 Day
Menu-Driven System	Interactive text menu in main method	1 Day
Search Cars by Manufacturer and Type	Traverse linked list, match brand and type	1 Day

Car Wash Queue System	Enqueue/Dequeue cars using linked list	1 Day
Sell Car	Remove car node from linked list and HashMap	1 Day

UML

- Car (abstract)
    - String brand
    - String type
    - int year
    - double price
    - int x, y
    - +displayInfo()
- BasicCar : Car
- PremiumCar : Car
- Node
    - Car carObj
    - Node next
- VendingMachine
    - Node head
    - HashMap<String, Car>
    - +addCar()
    - +retrieveCar()
    - +showVendingMachine()
    - +sortCar()
    - +sellCar()
- CarWash
    - Node head

- +enQueue()
- +processQueue()

Pre/Post Conditions

Precondition	Action	Postcondition
Empty Vending Machine	Load cars from file	Cars added at valid slots
Slot Occupied	Try to add car to same slot	Print error and prevent overwrite
Car Found by Search	Retrieve car from position	Print car details
Car Wash Queue Empty	Process queue	Print "Queue is empty."
Car Sold	Remove from list and HashMap	Car deleted successfully

---

Pseudocode/Design

VendingMachine addCar(Node newNode)

```
if head == null:
    head = newNode
else:
    traverse linked list
    if newNode.x and y match current node:
        print error
        return
    if end of list:
        add newNode
add car to HashMap with key (x,y)
```

VendingMachine retrieveCar(x, y)

```
build key from x and y
search key in HashMap
if found:
    return Car
else:
    return null
```

VendingMachine sortCar(choice)

```
move all cars into ArrayList
if choice == 1:
   sort by price
else:
   sort by year
print sorted cars
```

CarWash enQueue(Car newCar)

```
create Node with newCar
add to end of queue
print updated queue
```

CarWash processQueue()

```
while head is not null:
   print dequeued car
   move head to next node
print "Queue is empty"
```

Final Notes

This updated design now fully uses **LinkedList**, **HashMap**, **ArrayList**, and **Queue** in proper ways. Object-oriented design was applied through abstraction and inheritance.

- HashMap: Fast car lookup by position
- Queue: Process cars for washing in order
- ArrayList: Sort inventory efficiently
- LinkedList: Dynamic car storage without size limitation

The system is now dynamic and modular